Novopayment - [BNP] [INTERNO] Análisis de reutilización de SendPassCode para 3DS

NovoPayment, Inc.

Version v1.0.0, 17.03.2026: [BNP] [INTERNO] Análisis de reutilización de SendPassCode para 3DS Documentation

Table of Contents

1. Objetivo . . . 1
   1.1. Contexto del análisis interno . . . 1
   1.2. Contexto . . . 1
   1.3. Alcance del análisis . . . 1
   1.4. Criterios de aceptación de la historia . . . 1
2. Resumen ejecutivo . . . 3
3. Referencia de flujo . . . 4
   3.1. Convención visual del comparativo . . . 6
4. Hallazgos del comportamiento actual . . . 7
   4.1. Funcionamiento end-to-end (E2E) . . . 7
5. Plataformas de envío (email/sms) . . . 9
6. Dependencias y restricciones fuera de tokenization . . . 10
7. Análisis contra la necesidad de 3DS . . . 11
8. Evidencia técnica del endpoint actual . . . 12
   8.1. Endpoint . . . 12
   8.2. Path params . . . 12
   8.3. Request body . . . 12
   8.4. Alineación con referencia externa (SendPasscode) . . . 13
      8.4.1. Ejemplo de request . . . 14
   8.5. Contenido esperado dentro de `encryptedData` (desencriptado) . . . 14
   8.6. Responses . . . 15
      8.6.1. 200 OK (flujo normal) . . . 15
      8.6.2. 200 OK (body invalido tras decrypt) . . . 15
      8.6.3. Error de negocio (`NovoException`) . . . 15
      8.6.4. 500 Internal Server Error . . . 16
9. Evaluación de factibilidad para 3DS . . . 17
   9.1. Pros de una reutilización parcial . . . 17
   9.2. Contras y limitaciones . . . 17
   9.3. Riesgos de reutilizarlo directamente en 3DS . . . 18
   9.4. cURL de ejemplo . . . 18
   9.5. cURLs de referencia observada . . . 18
      9.5.1. SendPassCode (proceso actual según referencia externa) . . . 18
   9.6. Por que no es reutilizable de forma directa . . . 21
   9.7. Que si se puede reutilizar . . . 21

9.8. Recomendación 22
9.9. Clasificación final de la recomendación 22
9.10. Validación contra criterios de aceptación 22

Chapter 1. Objetivo

Documentar, con base en la historia interna y en la implementación actual, si el endpoint `POST /{tenant}/{customerType}/tokenization/sendPassCode` puede reutilizarse en el servicio de 3DS o si requiere un endpoint o flujo dedicado.

1.1. Contexto del análisis interno

Item	Detalle
Historia	`[BNP] [INTERNO] Análisis de reutilización de SendPassCode para 3DS`
Ticket origen	`CEB-4255: [BNP] / pa-bnp / 3DS / SendOTP: Incluir en el proceso de envío de Mensajes Email & SMS del Banco, las peticiones de SendOTP`
Objetivo del análisis	Determinar si `SendPassCode` puede reutilizarse en el servicio de 3DS o si se requiere un endpoint o flujo específico.
Referencia funcional externa	https://developer.novopayment.com/api/tokenization/issuer-tokenization-api-v1.0#/operations/SendPasscode

1.2. Contexto

Se solicitó evaluar si el endpoint `SendPassCode`, hoy implementado en el contexto de `tokenización`, puede cubrir la necesidad de envío de código y validación requerida por 3DS.

1.3. Alcance del análisis

Este documento cubre:

- Objetivo funcional actual del endpoint y del flujo donde se usa hoy.
- Dependencias técnicas y de negocio necesario para operarlo.
- Validación de capacidades relevantes para 3DS: canal de envío, expiración, control de intentos y trazabilidad.
- Impactos, riesgos y limitaciones de reutilizar el flujo actual en 3DS.
- Recomendación técnica final: reutilizable sin cambios, reutilizable con ajustes o no reutilizable.

1.4. Criterios de aceptación de la historia

- Se documenta el comportamiento actual de `SendPassCode` y su alcance real.
- Se documentan las dependencias y restricciones para usarlo fuera de tokenization.

- Se entrega una recomendación técnica clara sobre su reutilización en 3DS, con pros, contras y riesgos.

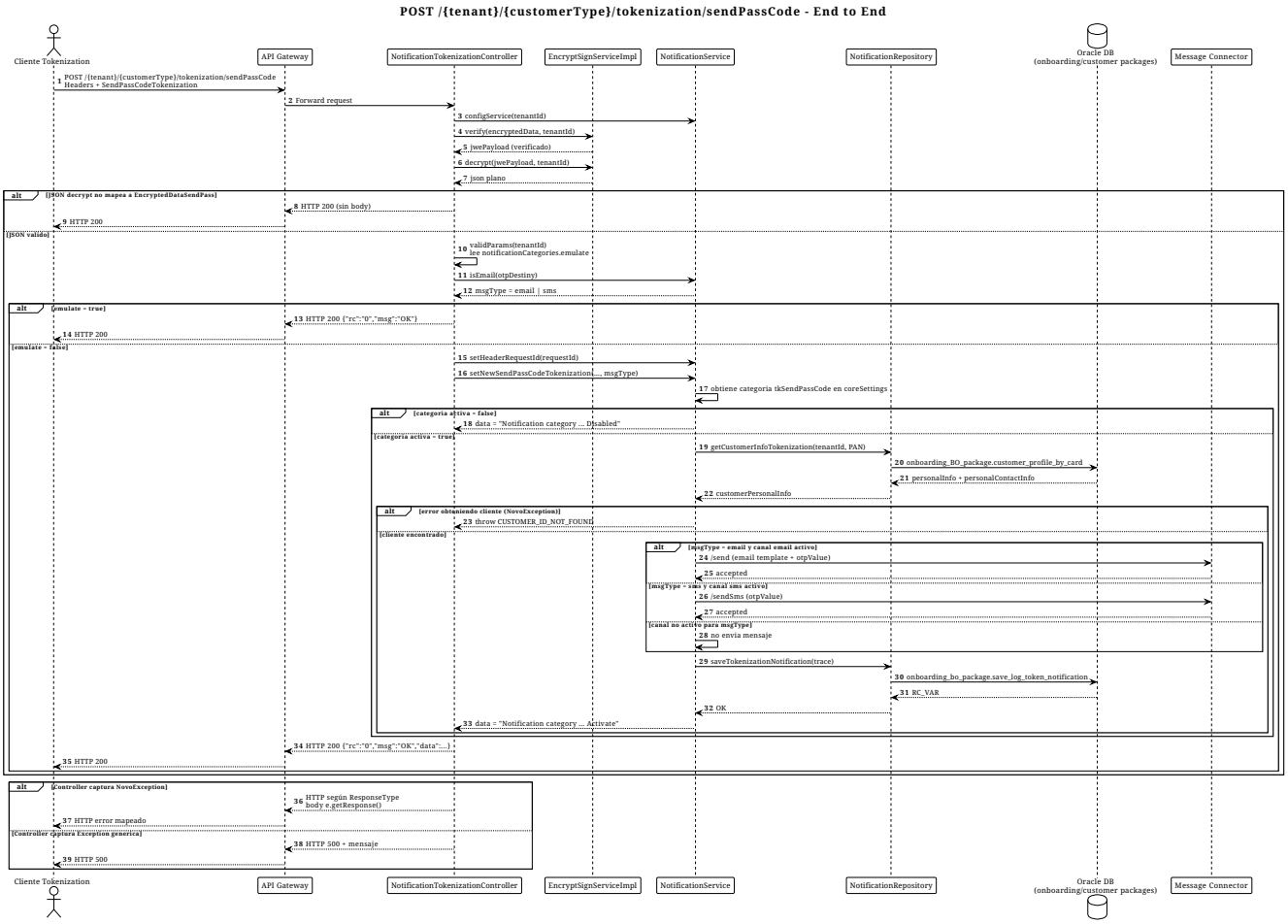
Chapter 2. Resumen ejecutivo

Punto	Conclusión
Decisión principal	`SendPassCode` **no debe reutilizarse directamente** como endpoint de negocio para 3DS.
Reutilización viable	**Parcial**, solo a nivel de componentes internos (criptografía, selección de canal e integración con `Message Connector`).
Motivo principal	El flujo actual está acoplado al dominio de `tokenización` en contrato, categoría, lookup y persistencia.
Recomendación	Crear un endpoint o flujo específico para 3DS y reutilizar internamente solo componentes comunes.

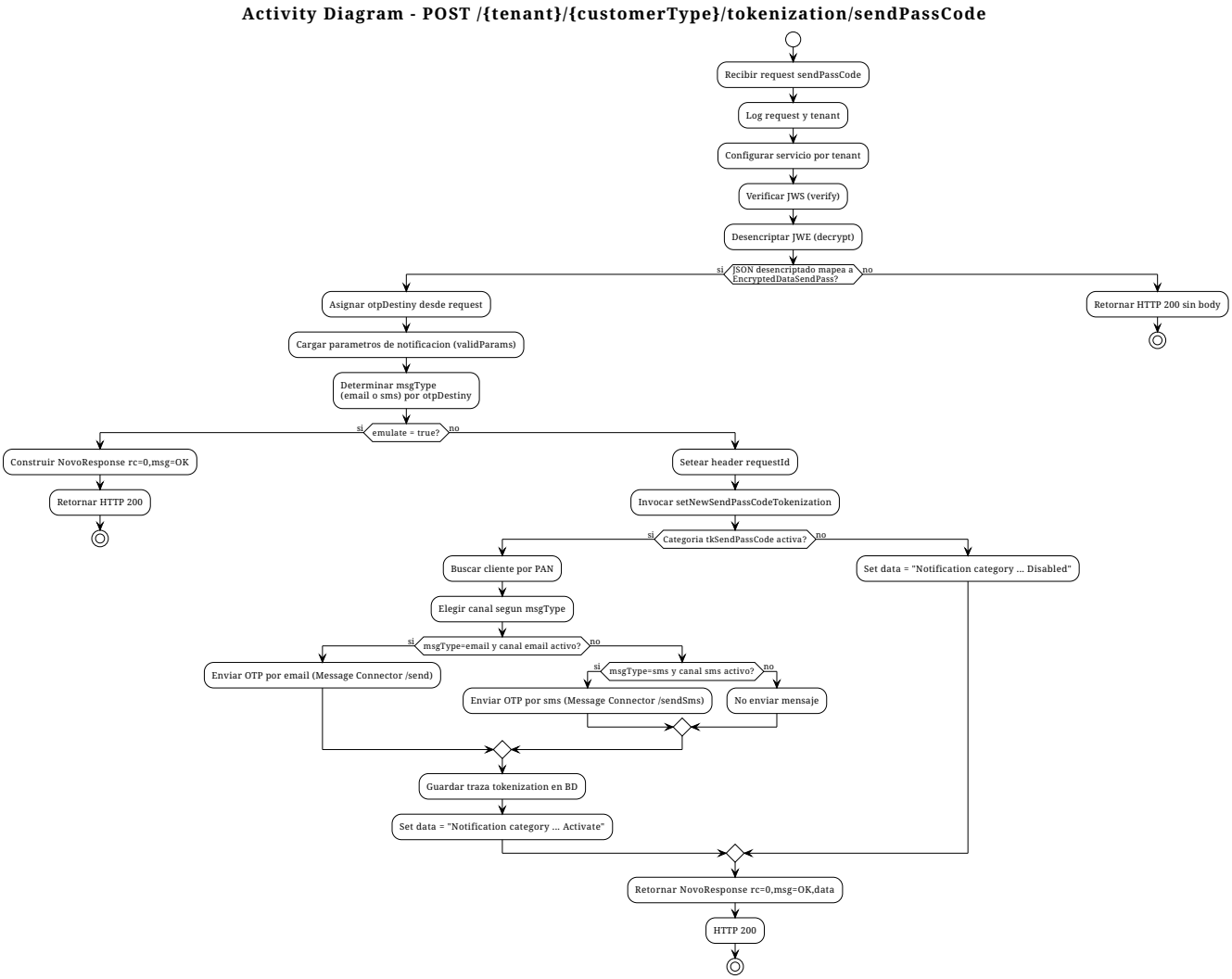
Chapter 3. Referencia de flujo

Este endpoint tiene dos vistas complementarias de flujo:

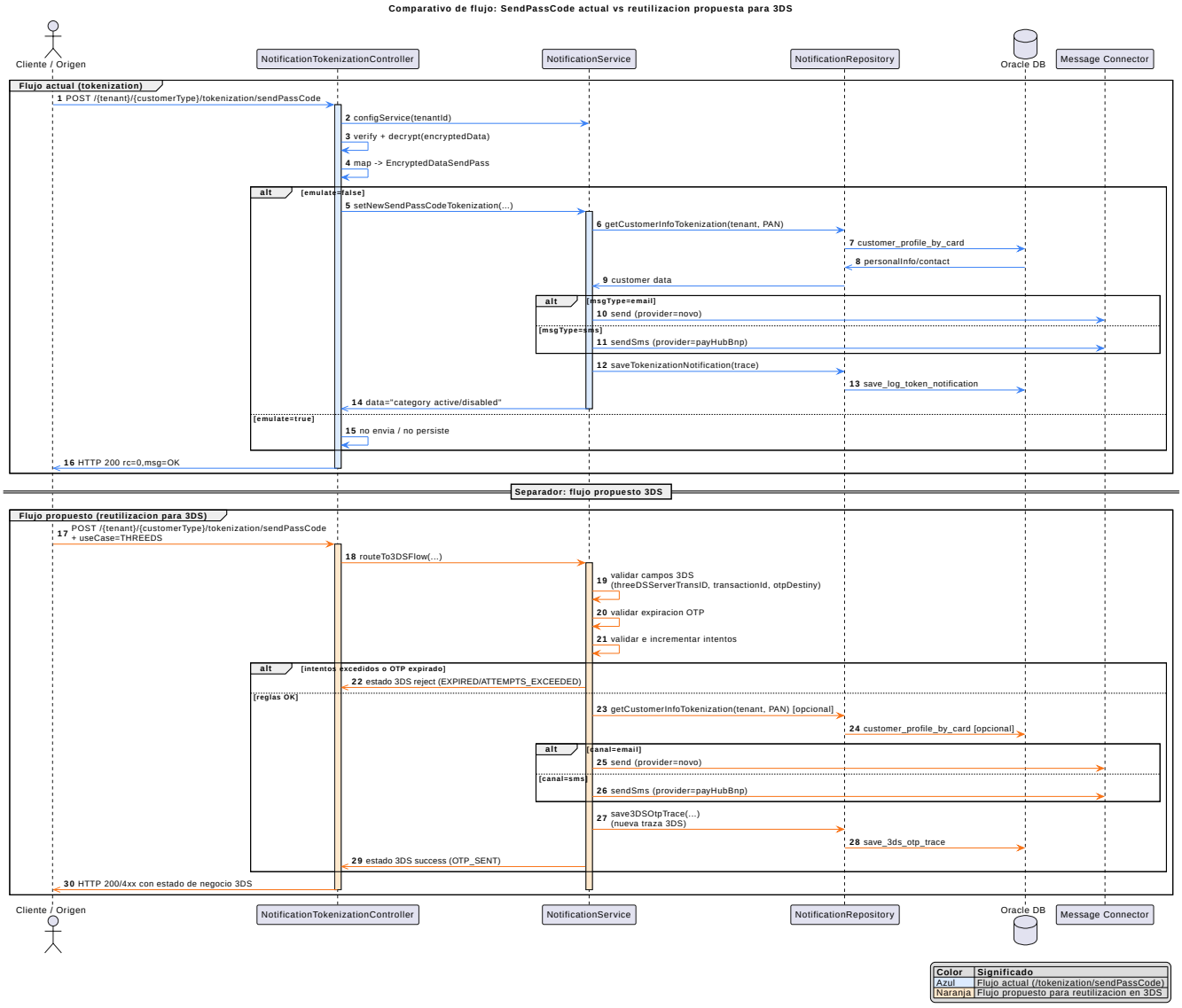
1. Secuencia end-to-end: `docs/adoc/CEB/diagrams/5371.puml`



1. Diagrama de actividad (flujo puro): `docs/adoc/CEB/diagrams/flow/5371.puml`



1. Diagrama comparativo (actual vs propuesta 3DS): `docs/adoc/CEB/diagrams/CEB-5371-3DS.puml`



3.1. Convención visual del comparativo

1. Azul: flujo actual de `sendPassCode` en tokenization.
2. Naranja: flujo propuesto para reutilización controlada en 3DS.

Chapter 4. Hallazgos del comportamiento actual

4.1. Funcionamiento end-to-end (E2E)

El flujo funcional de `POST /{tenant}/{customerType}/tokenization/sendPassCode` es:

1. El controlador recibe `SendPassCodeTokenization`, registra logs y carga configuración por tenant con `configService(tenantId)`.
2. Se valida y procesa `encryptedData` con `verify(…)` y luego `decrypt(…)`.
3. El JSON desencriptado se mapea a `EncryptedDataSendPass`.
   a. Si el mapeo falla, responde `HTTP 200` sin body.
4. Se calcula el canal de envío (`msgType`) desde `otpDestiny`:
   a. Si `isEmail(otpDestiny)=true`, canal `email`.
   b. En caso contrario, canal `sms`.
5. Se evalúa `notificationCategories.emulate`:
   a. Si `emulate=true`, responde `rc=0`, `msg=OK` sin envío real ni persistencia.
   b. Si `emulate=false`, continua el flujo real.
6. En flujo real se invoca `setNewSendPassCodeTokenization(…)`.
7. El servicio valida la categoría `tkSendPassCode`:
   a. Si está inactiva, retorna el mensaje `Notification category: tkSendPassCode Disabled`.
   b. Si está activa, continua.
8. Se consulta cliente por PAN con `getCustomerInfoTokenization(…)` en repositorio.
   a. Si ocurre `NovoException` en esta consulta, se lanza `CUSTOMER_ID_NOT_FOUND`.
9. Se envía OTP según canal y configuración activa de categoría:
   a. Email → `sendEmailSendPassCodeTokenization` → provider `novo`.
   b. SMS → `sendSmsSendPasscodeTokenization` → provider `payHubBnp`.
   c. Si el canal correspondiente no está activo en configuración, no se envía mensaje.
10. Se guarda traza de `tokenización` en BD con `saveTokenizationNotification(…)`.
    a. Si falla el guardado de traza, se registra en log y el flujo principal continua.
11. Se responde `HTTP 200` con `NovoResponse` (`rc=0`, `msg=OK`) y `data` con estado de categoría.

Manejo de errores del endpoint:

1. `NovoException`: responde con HTTP según `ResponseType` y body de negocio.
2. Excepción genérica: responde con `HTTP 500` y mensaje de error.

Chapter 5. Plataformas de envío (email/sms)

La selección del canal para `sendPassCode` se define por `otpDestiny`:

1. Si `otpDestiny` es email (`isEmail=true`), se usa el canal `email`.
2. En caso contrario, se usa el canal `sms`.

En `sendPassCode` los providers quedan así:

Flujo	Canal	Método en servicio	Provider (`$PROVIDERS$`)
`/tokenization/sendPassCode`	Email	`sendEmailSendPassCodeTokenization`	`novo`
`/tokenization/sendPassCode`	SMS	`sendSmsSendPasscodeTokenization`	`payHubBnp`

Comparativo rápido con otros flujos del mismo servicio:

Flujo	Canal	Método en servicio	Provider (`$PROVIDERS$`)
`/tokenization/notifications`	Email	`sendEmailNotificationTokenization`	`novo`
`/tokenization/notifications`	SMS	`sendSmsNotificationTokenization`	`payHubBnp`
Notificaciones generales	Email	`sendEmail`	`""` (sin provider explicito)
Notificaciones generales	SMS	`sendSms`	`twilio`

El provider real final depende del `ruteo` y la configuración del Message Connector; aquí se documenta el valor que este servicio `setea` en la URL.

Chapter 6. Dependencias y restricciones fuera de tokenization

El uso actual de `sendPassCode` depende de componentes y supuestos propios del dominio de `tokenización`:

1. Configuración por tenant (`configService`) y categoría fija `tkSendPassCode`.
2. Validación criptográfica basada en `verify(…)` y `decrypt(…)` con `tokenizationSettings`.
3. Contrato de entrada `SendPassCodeTokenization`.
4. Mapeo del payload desencriptado a `EncryptedDataSendPass`.
5. Consulta de cliente por PAN mediante `getCustomerInfoTokenization(…)`.
6. Persistencia de auditoría mediante `saveTokenizationNotification(…)`.
7. Envío por `Message Connector` con providers actuales `novo` y `payHubBnp`.

Estas dependencias implican que el endpoint no es un servicio genérico de OTP, sino un flujo especializado de `tokenización`.

Chapter 7. Análisis contra la necesidad de 3DS

La historia solicita validar si el comportamiento actual de `SendPassCode` cubre la necesidad de envío de código y validación para 3DS. El resultado del análisis es el siguiente:

1. **Canal de envío**: cubierto parcialmente; el endpoint ya decide entre email y SMS usando `otpDestiny`.
2. **Expiración**: cubierta solo como dato transportado o persistido; no existe lógica de vencimiento.
3. **Control de intentos**: no cubierto; `otpMaxReached` no acciona reglas funcionales.
4. **Trazabilidad**: cubierta parcialmente; existe logging y persistencia, pero con semántica de `tokenización`.
5. **Dominio funcional 3DS**: no cubierto; faltan contrato, estados y trazabilidad propios de autenticación 3DS.

Por lo anterior, el endpoint actual sirve como **evidencia de capacidades reutilizables**, pero no como implementación final adecuada para 3DS.

Chapter 8. Evidencia técnica del endpoint actual

8.1. Endpoint

```
POST /{tenant}/{customerType}/tokenization/sendPassCode
Content-Type: application/json
Accept: application/json
```

8.2. Path params

Parámetro	Descripción
`tenant`	Identificador del banco/tenant (usado para configuración y llaves criptográficas).
`customerType`	Tipo de cliente requerido por el routing base del controlador.

8.3. Request body

El body corresponde a la entidad `SendPassCodeTokenization`.

Campo	Tipo	Descripción
`tokenRequestorID`	string	Identificador del token requestor.
`tokenReferenceID`	string	Referencia del token.
`panReferenceID`	string	Referencia del PAN.
`clientWalletAccountID`	string	Cuenta wallet del cliente.
`otpMaxReached`	string	Indicador/estado de máximos intentos OTP.
`otpMethodIdentifier`	string	Canal/método OTP.
`otpValue`	string	OTP a notificar.
`otpDestiny`	string	Destino OTP (email o teléfono). Si es email, el endpoint enruta por canal email; de lo contrario por sms.
`otpReason`	string	Motivo OTP.
`otpExpirationDate`	string	Fecha de expiración OTP.

Campo	Tipo	Descripción
`deviceInfo`	object	Objeto con metadata del dispositivo (se usa al persistir log).
`encryptedData`	string	Payload cifrado/firmado que al desencriptar debe mapear a `EncryptedDataSendPass`.

En la referencia externa de `SendPasscode` puede verse `encryptedData` como objeto JSON en claro. En esta implementación de `notification-core`, el endpoint procesa `encryptedData` como **string** (JWS/JWE), luego ejecuta `verify` y `decrypt` para mapearlo a `EncryptedDataSendPass`.

8.4. Alineación con referencia externa (SendPasscode)

El siguiente cuadro resume diferencias de contrato entre la referencia externa y la implementación actual evaluada:

Campo	Referencia externa	Implementación actual (`notification-core`)
`otpMaxReached`	Boolean (`true/false`)	Se maneja como `String` en entidad de entrada/salida interna.
`OtpReason`	Se muestra con `O` mayúscula en algunos ejemplos	Se recomienda enviar `otpReason` (camelCase) para consistencia de mapeo.
`encryptedData`	Objeto JSON en ejemplo	Se espera `String` firmado/cifrado; el objeto va dentro del contenido desencriptado.
`tokenInfo`	Puede verse al mismo nivel del request en ejemplo externo	En esta implementación, `tokenInfo` debe viajar en el payload interno de `encryptedData`.

Para pruebas de integración sobre este servicio, use el contrato operativo local (tabla anterior), aunque la fuente externa se mantenga como referencia funcional del dominio.

8.4.1. Ejemplo de request

```
{
  "tokenRequestorID": "50123456789",
  "tokenReferenceID": "TRX-9ad77f7f",
  "panReferenceID": "PAN-ff28a4",
  "clientWalletAccountID": "wallet-user-001",
  "otpMaxReached": "false",
  "otpMethodIdentifier": "SMS",
  "otpValue": "184392",
  "otpDestiny": "+573001112233",
  "otpReason": "TOKEN_PROVISIONING",
  "otpExpirationDate": "2026-03-17T21:10:00Z",
  "deviceInfo": {
    "deviceID": "A1B2C3D4E5"
  },
  "encryptedData": "<JWS que contiene JWE>"
}
```

8.5. Contenido esperado dentro de `encryptedData` (desencriptado)

Internamente, el controlador hace `verify` + `decrypt`, y el JSON resultante se mapea a `EncryptedDataSendPass`.

```
{
  "cardholderInfo": {
    "primaryAccountNumber": "4111111111111111",
    "name": "JOHN DOE",
    "otpDestiny": null,
    "highValueCustomer": false
  },
  "riskInformation": {
    "accountHolderName": "JOHN DOE",
    "visaTokenScore": "05",
    "visaTokenDecisioning": "APPROVE"
  },
  "tokenInfo": {
    "token": "4761739001010010",
    "tokenType": "NETWORK",
    "tokenStatus": "ACTIVE",
    "tokenAssuranceMethod": "RISK_BASED_AUTH",
    "numberOfActiveTokensForPAN": 1,
```

```
    "numberOfInactiveTokensForPAN": 0,
    "numberOfSuspendedTokensForPAN": 0,
    "autoFillIndicator": true
  }
}
```

8.6. Responses

8.6.1. 200 OK (flujo normal)

Cuando no hay excepción, la respuesta es `NovoResponse` serializada.

```
{
  "rc": "0",
  "msg": "OK",
  "data": "Notification category: tkSendPassCode Activate"
}
```

Notas:

1. Si la categoria `tkSendPassCode` esta inactiva, `data` cambia a `Notification category: tkSendPassCode Disabled`.
2. Si `notificationCategories.emulate=true`, se devuelve 200 con `rc/msg` y sin ejecutar envío/persistencia (normalmente `data` ausente).

8.6.2. 200 OK (body invalido tras decrypt)

Si falla el mapeo del JSON desencriptado a `EncryptedDataSendPass`, el controlador retorna `HTTP 200` sin body.

8.6.3. Error de negocio (`NovoException`)

Si se lanza `NovoException` (por ejemplo, cliente no encontrado), el endpoint responde con el HTTP code asociado a `ResponseType` y el body de error.

Ejemplo referencial de estructura:

```
{
  "rc": "<codigo_negocio>",
  "msg": "<descripción>"
}
```

8.6.4. 500 Internal Server Error

Ante excepción no controlada:

```
<mensaje de la excepcion>
```

Chapter 9. Evaluación de factibilidad para 3DS

Capacidad requerida por 3DS	Cobertura actual en `sendPassCode`	Observación
Canal de envío	Parcial	Si soporta email o SMS según `otpDestiny`, y reutiliza `Message Connector`.
Expiración OTP	Parcial	Recibe y persiste `otpExpirationDate`, pero no aplica validación ni gestion funcional de vencimiento.
Control de intentos	No	Recibe/persiste `otpMaxReached`, pero no implementa conteo, bloqueo, incremento ni reglas de intentos.
Trazabilidad	Parcial	Existe logging, `requestId` y persistencia, pero la traza queda modelada como tokenization, no como autenticación 3DS.
Contrato 3DS	No	`El request/response` y el payload desencriptado están orientados a tokenization, no a challenge/autenticación 3DS.

9.1. Pros de una reutilización parcial

1. Ya existe integración para envío OTP por email y SMS.
2. Ya existe infraestructura de `verify(…)` / `decrypt(…)`.
3. Ya existe lookup de cliente por PAN.
4. Ya existe integración centralizada con `Message Connector`.

9.2. Contras y limitaciones

1. El contrato actual está acoplado al dominio de tokenization.
2. La categoria funcional es fija (`tkSendPassCode`).
3. La persistencia actual registra eventos en estructura de tokenization.
4. El endpoint no implementa lógica completa de expiración, intentos ni validación OTP.
5. No contempla atributos y estados propios de 3DS.

9.3. Riesgos de reutilizarlo directamente en 3DS

1. Mezcla de dominios de negocio: tokenization y autenticación 3DS.
2. Auditoria y trazabilidad ambigua al persistir eventos 3DS como tokenization.
3. Incremento de acoplamiento y riesgo de regresión sobre el flujo actual de tokenization.
4. Necesidad de introducir cambios de negocio en un endpoint ya operativo para otro caso de uso.

9.4. cURL de ejemplo

```
curl -X POST "https://<host>/<tenant>/<customerType>/tokenization/sendPassCode" \
  -H "Content-Type: application/json" \
  -H "Accept: application/json" \
  -d '{
    "tokenRequestorID": "50123456789",
    "tokenReferenceID": "TRX-9ad77f7f",
    "panReferenceID": "PAN-ff28a4",
    "clientWalletAccountID": "wallet-user-001",
    "otpMaxReached": "false",
    "otpMethodIdentifier": "SMS",
    "otpValue": "184392",
    "otpDestiny": "+573001112233",
    "otpReason": "TOKEN_PROVISIONING",
    "otpExpirationDate": "2026-03-17T21:10:00Z",
    "deviceInfo": {"deviceID": "A1B2C3D4E5"},
    "encryptedData": "<JWS que contiene JWE>"
  }'
```

9.5. cURLs de referencia observada

9.5.1. SendPassCode (proceso actual según referencia externa)

```
curl --request POST \
  --url https://urlissuer/v1/sendPasscode \
  --header 'Content-Type: application/json' \
  --data '{
  "tokenRequestorID": 40022358888,
  "tokenReferenceID": "DNITHE30194726875288888",
  "panReferenceID": "V-3019231458906209630635",
  "clientWalletAccountID": "string",
  "otpMaxReached": true,
  "otpMethodIdentifier": "string",
```

```
  "otpValue": "string",
  "OtpReason": "PROVISIONING",
  "otpExpirationDate": "string",
  "deviceInfo": {
    "deviceID": "CTF0000000001",
    "deviceIndex": 99,
    "deviceLanguageCode": "eng",
    "deviceType": "PHONE",
    "deviceName": "Novo_Phone",
    "deviceNumber": 6506198963,
    "osType": "ANDROID",
    "osVersion": "4.4.4",
    "osBuildID": 13.1,
    "deviceIDType": "TEE",
    "deviceManufacturer": "Samsung",
    "deviceBrand": "Galaxy",
    "deviceModel": "M05",
    "deviceLocation": "string",
    "deviceIPAddressV4": "127.0.0.1",
    "locationSource": "WIFI",
    "tokenProtectionMethod": "CLOUD",
    "originalDeviceID": "...",
    "originalDeviceType": "..."
  },
  "encryptedData": {
    "cardholderInfo": {
      "primaryAccountNumber": "0000000000000000",
      "otpDestiny": "57300800800"
    }
  },
  "tokenInfo": {
    "token": "...",
    "tokenType": "ECOMMERCE",
    "tokenStatus": "ACTIVE",
    "tokenExpirationDate": {
      "month": 7,
      "year": 2016
    },
    "tokenRequestorName": "NovoPayment Wallet",
    "tokenAssuranceMethod": "00",
    "numberOfActiveTokensForPAN": 2,
    "numberOfInactiveTokensForPAN": 2,
    "numberOfSuspendedTokensForPAN": 0,
    "tokenActivationDate": "2022-04-25T23:19:12.000Z",
    "tokenDeactivationDate": "...",
    "lastTokenStatusUpdatedTime": "...",
```

```
    "originalToken": "...",
    "originalTokenRequestorID": "...",
    "originalTokenReferenceID": "...",
    "originalTokenType": "SECURE_ELEMENT",
    "originaltokenAssuranceMethod": "00",
    "autoFillIndicator": false
  }
}'
```

El cURL anterior corresponde al proceso actual de `SendPassCode` documentado en la referencia externa.

```
curl --location 'https://banco-nacional-pa.novopayment.net/notification-
core/v1/bnp_pa/retail/tokenization/sendPassCode' \
--header 'Content-Type: application/json' \
--header 'x-context-id: bnp_pa' \
--data-raw '{
  "tokenRequestorID": 40022358888,
  "tokenReferenceID": "DNITHE30194726875288888",
  "panReferenceID": "V-3019231458906209630635",
  "clientWalletAccountID": "string",
  "otpMaxReached": true,
  "otpMethodIdentifier": "string",
  "otpValue": "12345",
  "otpDestiny": "joaquinteromo552@gmail.com",
  "OtpReason": "PROVISIONING",
  "otpExpirationDate": "string",
  "deviceInfo": {
    "deviceID": "CTF0000000001",
    "deviceIndex": 99,
    "deviceLanguageCode": "eng",
    "deviceType": "PHONE",
    "deviceName": "Novo_Phone",
    "deviceNumber": 6506198963,
    "osType": "ANDROID",
    "osVersion": "4.4.4",
    "osBuildID": 13.1,
    "deviceIDType": "TEE",
    "deviceManufacturer": "Samsung",
    "deviceBrand": "Galaxy",
    "deviceModel": "M05",
    "deviceLocation": "string",
    "deviceIPAddressV4": "127.0.0.1",
    "locationSource": "WIFI",
```

```
    "tokenProtectionMethod": "CLOUD",
    "originalDeviceID": "...",
    "originalDeviceType": "..."
  },
  "encryptedData": "<payload omitido>"
}'
```

Los ejemplos anteriores son payloads de referencia observada y se documentan para contrastar el uso actual del endpoint con la necesidad de 3DS. == Factibilidad de reutilización para 3DS

Conclusión: **no es recomendable reutilizar este endpoint de forma directa para implementar 3DS**; **sí es factible reutilizar partes internas del flujo**.

9.6. Por que no es reutilizable de forma directa

Este endpoint está acoplado al caso de uso de tokenization/sendPassCode por varias razones:

1. El contrato de entrada está definido por `SendPassCodeTokenization`, con campos y semántica orientados a tokenization.
2. `El encryptedData` Se valida y se mapea a `EncryptedDataSendPass`, cuyo contenido esperado también responde al dominio de tokenization.
3. La categoria funcional usada por el servicio es fija: `tkSendPassCode`.
4. La persistencia se realiza con `saveTokenizationNotification(…)`, que guarda una traza con estructura de tokenization y no una de autenticación 3DS.
5. La lógica actual determina canal solo a partir de `otpDestiny` (email v.s sms), pero no contempla atributos propios de 3DS como `challenge`, `transaction status`, `acs`, `ds`, `cavv`, `eci`, `transStatusReason` o correlación específica de autenticación.

9.7. Que si se puede reutilizar

Hay componentes que podrían reaprovecharse para una implementación 3DS:

1. Verificación y desencriptado (`verify` / `decrypt`) si 3DS usa el mismo esquema JWS/JWE y administración de llaves.
2. Resolución del canal de envío a partir del destino (`isEmail`).
3. Integración con `Message Connector` para envío por email o SMS.
4. Consulta de cliente por PAN, siempre que el caso de uso 3DS mantenga esa forma de identificación.

9.8. Recomendación

Para 3DS, lo aconsejable es crear un **endpoint y contrato propios**, por ejemplo una categoria dedicada y un modelo de request/response alineado al dominio 3DS. Internamente, se pueden reutilizar piezas comunes del flujo actual (criptografía, selección de canal y adaptación al `Message Connector`), pero evitando reutilizar sin cambios el endpoint `sendPassCode`, porque mezclaría dos dominios de negocio distintos: **tokenization** y **autenticación 3DS**.

9.9. Clasificación final de la recomendación

Opción	Resultado
Reutilizable sin cambios	No
Reutilizable con ajustes	Solo a nivel de componentes internos, no como endpoint de negocio
No reutilizable / requiere endpoint dedicado	Si

9.10. Validación contra criterios de aceptación

1. **Criterio 1 cumplido**: se documentó el comportamiento actual de `SendPassCode` y su alcance real.
2. **Criterio 2 cumplidos**: se documentaron dependencias y restricciones para usarlo fuera de tokenization.
3. **Criterio 3 cumplidos**: se entrega una recomendación técnica clara con pros, contras, limitaciones y riesgos.